

OS_Proj1

Scheduling Simulation

모바일 시스템 공학과

32201345 나웅철

32217259 문서영

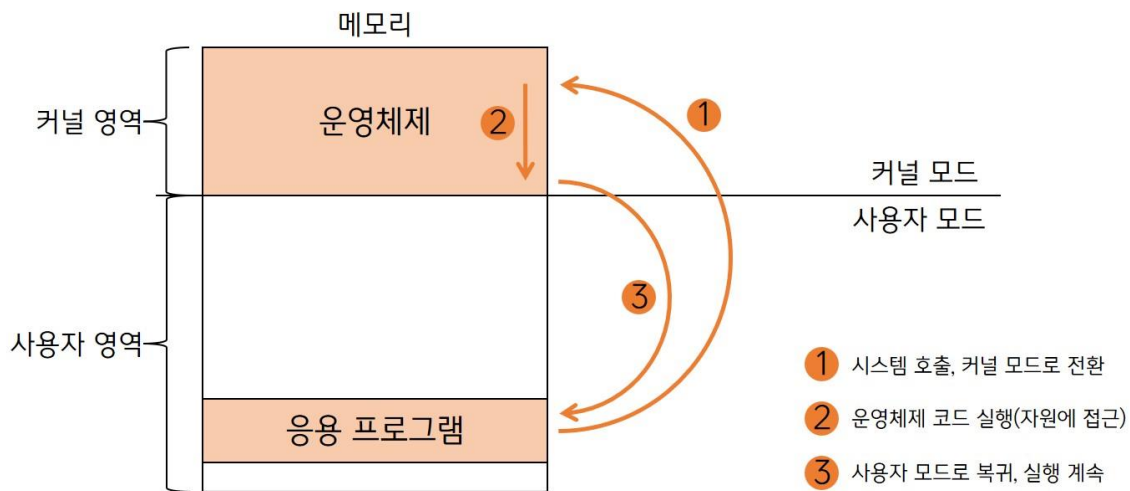
목차

- 프로세스 상태와 시스템 호출 with 이중모드
- 프로세스 계층 구조
- 프로세스의 메모리 영역 with PCB
- CPU 스케줄링
- 스케줄링 프로그램 설계 및 구현 계획
- 스케줄링 프로그램 구현
 - IPC 통신 _ Message Passing 방식 실습 _ chat.c
 - 시그널 사용 실습 _ ipc.c
 - 라운드 로빈 스케줄링 방식의 스케줄러 프로그램 구현 _ sec.c

Intro.

이번 과제는 스케줄링 시뮬레이션을 구현해보는 것으로 여러가지 개념들에 대한 이해가 필요하다. 스케줄링을 이해하기 위해서는 이중모드, 스케줄링의 대상이 되는 멀티 프로세스, 스케줄링의 기준이 되는 CPU 버스트와 I/O버스트, 스케줄링의 여러 기법들, 프로세스 상태, 프로세스 계층 구조 등등에 대한 사전적인 이해가 필요하다. 따라서 먼저 기본적으로 알아야 하는 지식들에 대해 먼저 다루고 그 후 시뮬레이터를 어떻게 구현할 지 설계하고 실제로 어떻게 구현했는지, 그 결과는 어떠한 지 작성할 계획이다.

프로세스 상태와 시스템 호출 feat 이중모드



[그림 1] 운영체제의 이중모드 구조

먼저 기본적으로 이번 과제는 부모프로세스와 자식프로세스 10개를 이용하되, 부모프로세스를 커널로서 작동하게 하고 자식프로세스들은 별개의 프로세스로 작동하게 하는 컨셉을 가지고 있다. 따라서 기본적으로 커널과 응용 프로그램이 어떤 구조를 지니고 있는지 부터 간단하게 살펴보고자 한다.

컴퓨터 시스템은 안전과 자원 보호를 위해 이중 모드를 지원한다. 이중 모드는 [그림 1]에서 보이는 것과 같이 사용자모드(User Mode)와 커널 모드(Kernel Mode)로 나뉘며, 각 모드에서는 수행할 수 있는 작업에 차이가 있다.

일단 프로세스마다 가상메모리가 할당되고, 이러한 가상메모리주소들은 실제메모리주소(물리메모리)와는 차이가 있음을 미리 알아야 한다. 사용자 모드와 달리 커널 모드의 경우 실제 하드웨어와 시스템 자원을 직접 처리 할 수 있는데 이러한 차이를 두는 이유는 실제 메모리에 접근하여 데이

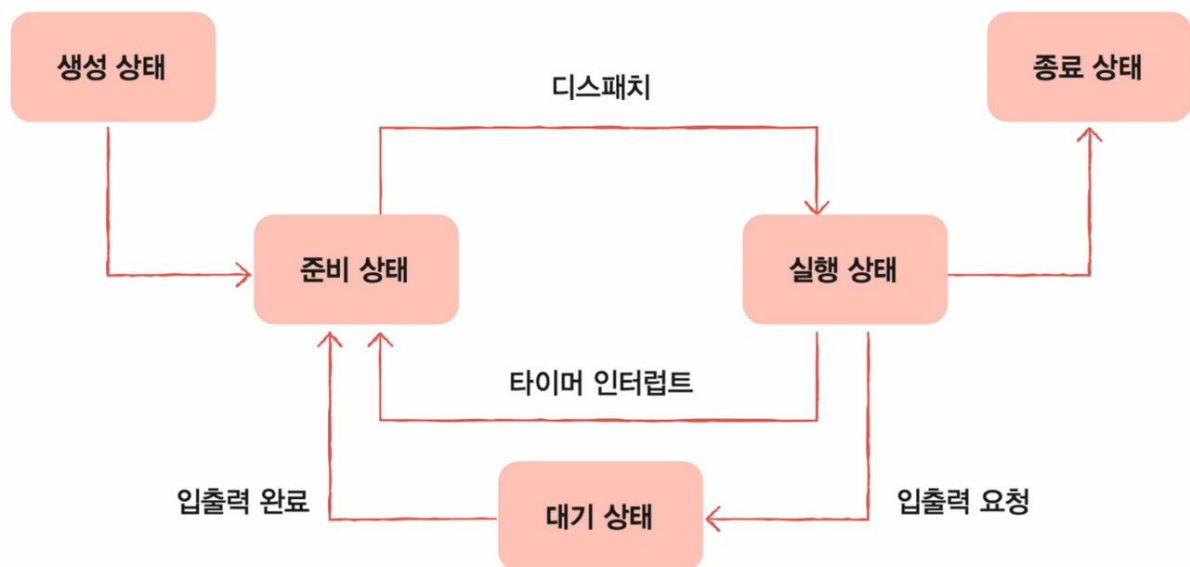
터를 다룰 경우 프로세스 간 간섭 또는 충돌이 일어날 수 있기 때문이다.

그렇다면 이중모드와 스케줄러는 어떤 관계를 가지고 있을까? 스케줄러는 CPU를 효율적으로 배분하기 위해 각 프로세스의 상태를 감시하고 관리한다. 스케줄러는 커널 모드에서 작동하며, 시스템 호출에 의해 CPU 제어권이 사용자 모드에서 커널 모드로 전환될 때 개입할 수 있다. 이때, 스케줄러는 프로세스의 우선순위와 CPU 할당 여부를 결정하며, 필요한 경우 다음 프로세스를 준비 상태에서 실행 상태로 전환해 CPU를 할당한다.

주요 프로세스 상태

- 생성 상태
 - 프로그램이 메모리에 적재되어 PCB를 할당 받은 상태
- 준비 상태
 - 당장이라도 CPU를 할당 받아 실행 가능 _ but, 자신의 차례가 올 때 까지 기다리는 상태
- 실행 상태
 - CPU를 할당 받아 실행 중인 상태 , (타이머 인터럽트 -> 준비상태), (입출력 인터럽트 -> 대기 상태)
- 대기 상태
 - 프로세스가 실행 도중 입출력장치를 사용하는 경우 -> 이후 입출력 완료 인터럽트를 받으면, -> 준비상태
- 종료 상태
 - 프로세스가 종료된 상태, PCB 프로세스의 메모리 영역 정리

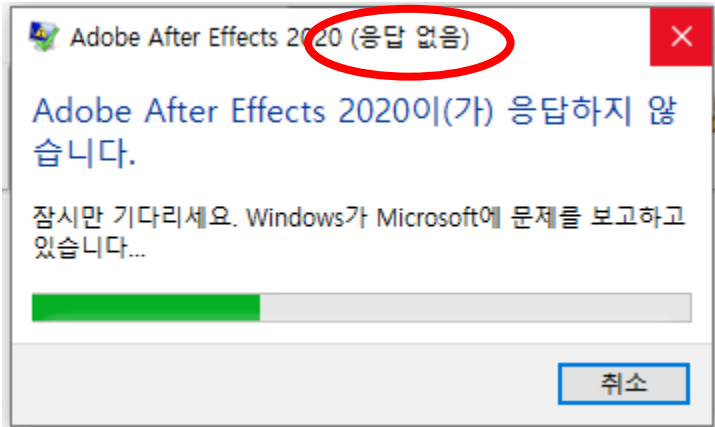
따라서 이번 과제에서 사용할 부모 프로세스는 커널로서 작동하기 위해 자식 프로세스들의 상태를 감시하고 관리하는 역할을 할 것이며, 자식 프로세스들의 상태를 결정하고, 시스템 호출의 기능도 가질 것이다. 자식 프로세스의 경우 일종의 프로그램으로서 부모 프로세스에게 자신의 상태를 보내 해당 정보를 토대로 부모프로세스에게 관리 받을 것이다. 중요한 단어 두 가지가 나왔는데 바로 프로세스의 상태와 시스템 호출이다.



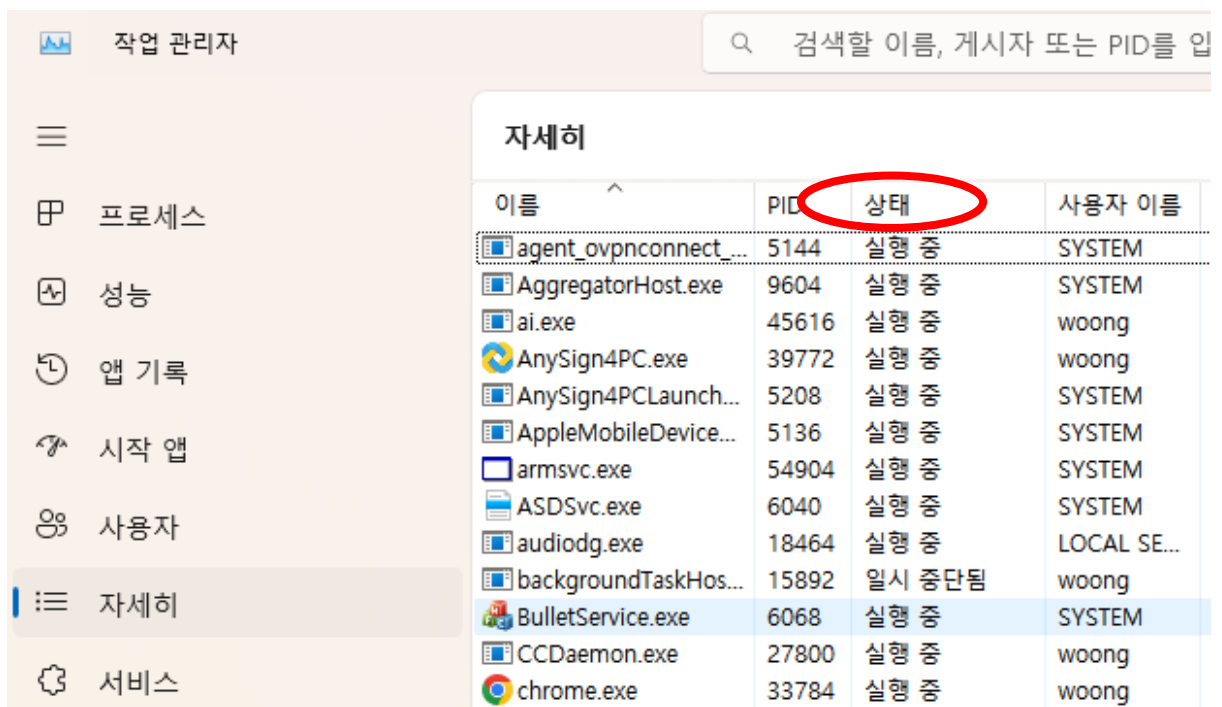
[그림 2] 프로세스 상태

프로세스는 존속기간 동안 상태를 갖는다. 기본적으로 [그림 2]에서 볼 수 있듯 5가지 상태를 말할 수 있는데 그 중 준비 상태, 실행 상태, 대기 상태가 가장 중요하다고 말할 수 있으며, cpu 작업과 i/o작업으로 분류할 수도 있다. 먼저 실행(Running)상태는 프로세스가 CPU를 점유하여 명령어를 실행 중인 상태이다. 그리고 준비(Ready)상태는 프로세스가 실행될 준비가 되었지만, CPU가 다른 작업을 수행 중이라 대기하는 상태이며, 마지막으로 대기(Waiting)상태는 프로세스가 I/O작업 등 특정 이벤트의 완료를 기다리는 상태로, 이벤트가 발생하면 준비상태로 돌아간다.

프로세스 상태 전환은 CPU 스케줄러에 의해 결정된다. 스케줄러는 프로세스의 우선순위와 스케줄링 정책에 따라 상태를 전환하며, 시스템 성능을 극대화하기 위해 최적의 프로세스를 선택해 CPU를 할당한다. 이번 프로젝트에서는 Round-Robin이라는 스케줄러 정책에 따라 구현할 예정임을 미리 알린다.



[그림 3] 상태_1



[그림 4] 상태_2

종류	시스템 호출	설명
프로세스 관리	fork()	새 자식 프로세스 생성
	execve()	프로세스 실행(메모리 공간을 새로운 프로그램의 내용으로 덮어쓰임)
	exit()	프로세스 종료
	waitpid()	자식 프로세스가 종료할 때까지 대기
파일 관리	open()	파일 열기
	close()	파일 닫기
	read()	파일 읽기
	write()	파일 쓰기
	stat()	파일 정보 획득
디렉터리 관리	chdir()	작업 디렉터리 변경
	mkdir()	디렉터리 생성
	rmdir()	비어 있는 디렉터리 삭제
파일 시스템 관리	mount()	파일 시스템 마운트
	umount()	파일 시스템 마운트 해제

[그림 5] 시스템 호출(System Call)

사용자 프로그램은 일반적으로 하드웨어 자원에 직접 접근할 수 없다. 대신, 시스템 호출이라는 인터페이스를 통해 OS의 기능을 호출한다. 즉 시스템 호출을 통해 커널모드를 사용할 수 있다고 할 수 있다. 좀 더 정확히 말하자면 시스템 호출은 사용자 모드에서 실행 중인 프로그램이 운영체제의 특권 기능을 사용할 때 커널 모드로 전환하기 위한 중요한 메커니즘인 것이다. 그렇다면 사용자 프로그램을 이루는 명령어들은 사용자 모드 명령어와 커널 모드 명령어로 이루어져 있다는 것을 알 수 있다. 그렇다면 해당 명령어를 인출하여 받아들이는 CPU는 해당 명령어가 커널 모드 전용인지 일반 사용자 모드 전용인지 어떻게 알 수 있을까? CPU가 명령어들을 인출하여 작업을 하다가 시스템 호출이 요청이 되면 레지스터 플래그 중 슈퍼 바이저 플래그가 0에서 1로 전환되어 커널 모드로 전환되고 해당 시점의 명령어는 커널 영역에서 진행하게 된다. 정확히 말하면 명령어 인출 전 레지스터 플래그 중 슈퍼바이저 플래그를 확인하고 모드를 결정한다.

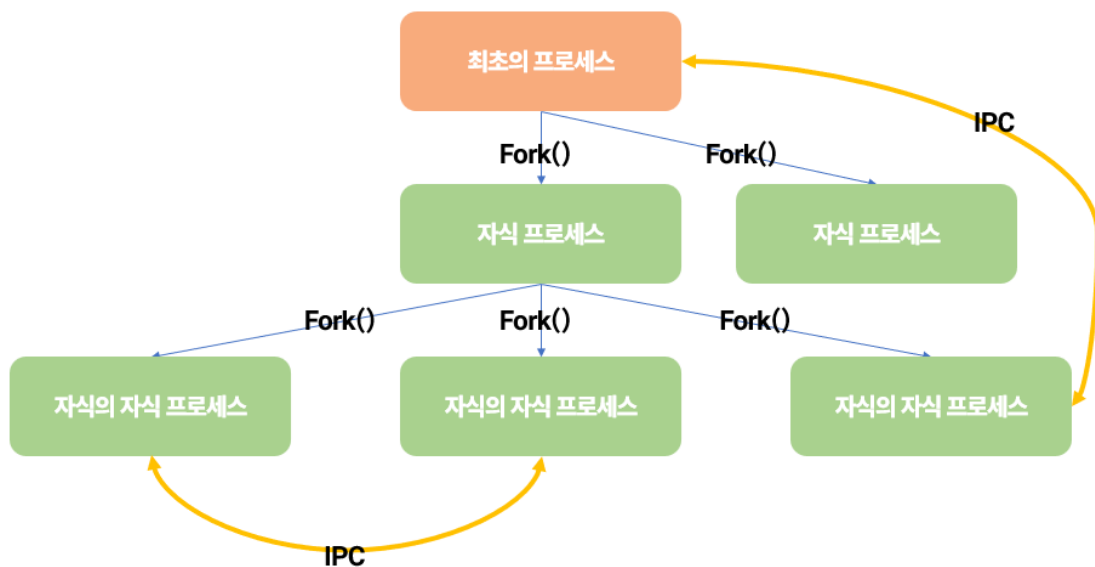
플래그 레지스터

부호 플래그	제로 플래그	캐리 플래그	오버플로우 플래그	인터럽트 플래그	슈퍼바이저 플래그
--------	--------	--------	-----------	----------	-----------

플래그 종류	의미	사용 예시
부호 플래그	연산한 결과의 부호를 나타낸다.	부호 플래그가 1일 경우 계산 결과는 음수, 0일 경우 계산 결과는 양수를 의미한다.
제로 플래그	연산 결과가 0인지 여부를 나타낸다.	제로 플래그가 1일 경우 연산 결과는 0, 0일 경우 연산 결과는 0이 아님을 의미한다.
캐리 플래그	연산 결과 올림수나 빌림수가 발생했는지를 나타낸다.	캐리 플래그가 1일 경우 올림수나 빌림수가 발생했음을 의미하고, 0일 경우 발생하지 않았음을 의미한다.
오버플로우 플래그	오버플로우가 발생했는지를 나타낸다.	오버플로우 플래그가 1일 경우 오버플로우가 발생했음을 의미하고, 0일 경우 발생하지 않았음을 의미한다.
인터럽트 플래그	인터럽트가 가능한지를 나타낸다. 인터럽트는 04-3절에서 설명한다.	인터럽트 플래그가 1일 경우 인터럽트가 가능함을 의미하고, 0일 경우 인터럽트가 불가능함을 의미한다.
슈퍼바이저 플래그	커널 모드로 실행 중인지, 사용자 모드로 실행 중인지를 나타낸다. 커널 모드와 사용자 모드는 09장에서 설명한다.	슈퍼바이저 플래그가 1일 경우 커널 모드로 실행 중임을 의미하고, 0일 경우 사용자 모드로 실행 중임을 의미한다.

[그림 6] 플래그 레지스터

프로세스 계층 구조



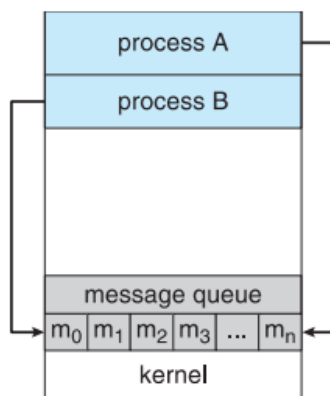
[그림 7] 프로세스 계층 구조

```
nwc-laptop@LAPTOP-RILL6KSB:~$ pstree
systemd--2*[agetty]
      |
      |--cron
      |--dbus-daemon
      |--init-systemd(Ub
      |   |--SessionLeader--Relay(437)--bash--pstree
      |   |--init--{init}
      |   |--login--bash
      |   |--{init-systemd(Ub}
      |--rsyslogd--3*[{rsyslogd}]
      |--systemd--(sd-pam)
      |--systemd-journal
      |--systemd-logind
      |--systemd-resolve
      |--systemd-timedat
      |--systemd-timesyn--{systemd-timesyn}
      |--systemd-udev--2*[{udev-worker}]
      |--unattended-upgr--{unattended-upgr}
      |--wsl-pro-service--7*[{wsl-pro-service}]
nwc-laptop@LAPTOP-RILL6KSB:~$ |
```

[그림 8] 우분투 상에서 살펴본 프로세스 트리

운영 체제에서 프로세스 계층 구조는 프로세스 간의 관계를 나타내는 구조로, 각 프로세스는 부모-자식 관계를 통해 계층적인 구성을 이룬다. 이러한 구조는 운영체제가 프로세스를 효과적으로 관리하고 제어할 수 있게 도와준다.

이전 과제인 멀티 스레드 과제에서 프로세스끼리 자원을 공유하지 않는다고 다뤘던 적이 있다. 하지만 [그림 7]에서 보이듯이 프로세스 간의 통신은 IPC기술을 통해 이루어진다. IPC는 Inter-Process Communication의 약자로 프로세스들 사이에 서로 데이터를 주고 받는 행위를 말한다. 이번 과제에서는 여러 IPC방식들 중 메시지 전달 방식을 사용할 예정이다.



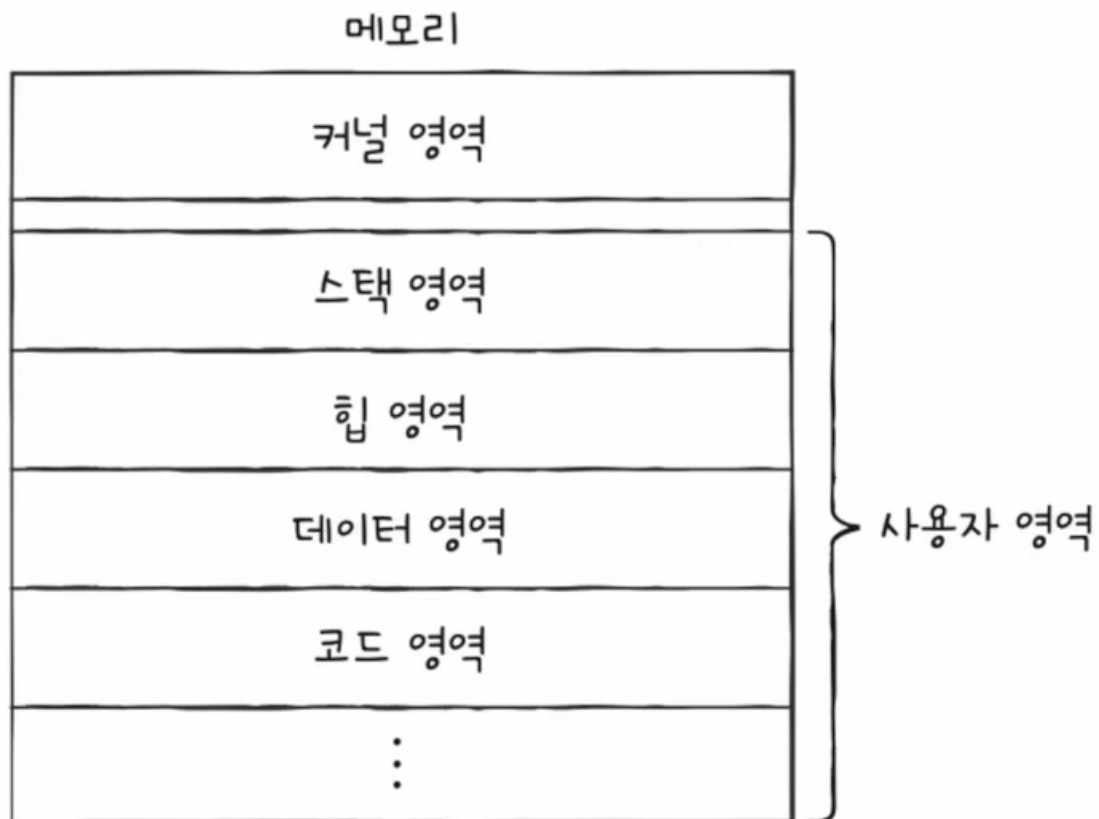
[그림 9] 메시지 전달 방식

메시지 전달 방식은 커널에 존재하는 공간에 메시지 큐를 만들어 프로세스들이 메시지를 저장할 공간을 만드는 방법이다. 이번 과제에서 필자는 이를 자식들이 자신의 상태를 부모 프로세스에게

전달하는 수단으로 사용할 계획이다.

프로세스의 메모리 영역과 PCB

이전 과제에서 다뤘던 내용으로 프로세스는 독립적인 가상 메모리 구조를 가진다고 하였다. 독립된다는 말은 구분된다는 뜻이고 따라서 식별할 수 있는 무엇인가가 필요한데 그것이 바로 PID(프로세스 아이디)이다. 또한 프로세스의 상태, 프로그램 카운터, CPU 레지스터와 상태 정보, 메모리 관리 정보 등등의 요소들이 프로세스를 관리하기 위해 사용된다. PCB의 경우 가상 메모리 구조 중 커널 영역에 저장되며, 프로세스의 경우 사용자 영역에 저장된다. 운영체제는 PCB의 내용을 참조하여 프로세스를 관리하고 CPU의 제어를 넘겨주는 작업을 수행한다.



[그림 10] 메모리 구조

PCB에 저장된 프로그램 카운터, CPU 레지스터, 메모리 관리 정보 등을 이용하면 프로세스가 일시 중단되었다가 재개될 때 중단된 시점부터 이어서 실행할 수 있어 문맥 전환시에 사용된다. 또한 각 프로세스의 우선순위와 상태가 PCB를 통해 확인할 수 있기 때문에 스케줄러가 이를 바탕으로 CPU할당을 결정하게 된다. 필자는 이번 과제에서 PCB 구조체에 PID와 CPU_burst, IO_burst를 담

아 부모 프로세스에 전달하는 방식을 사용할 계획이다.

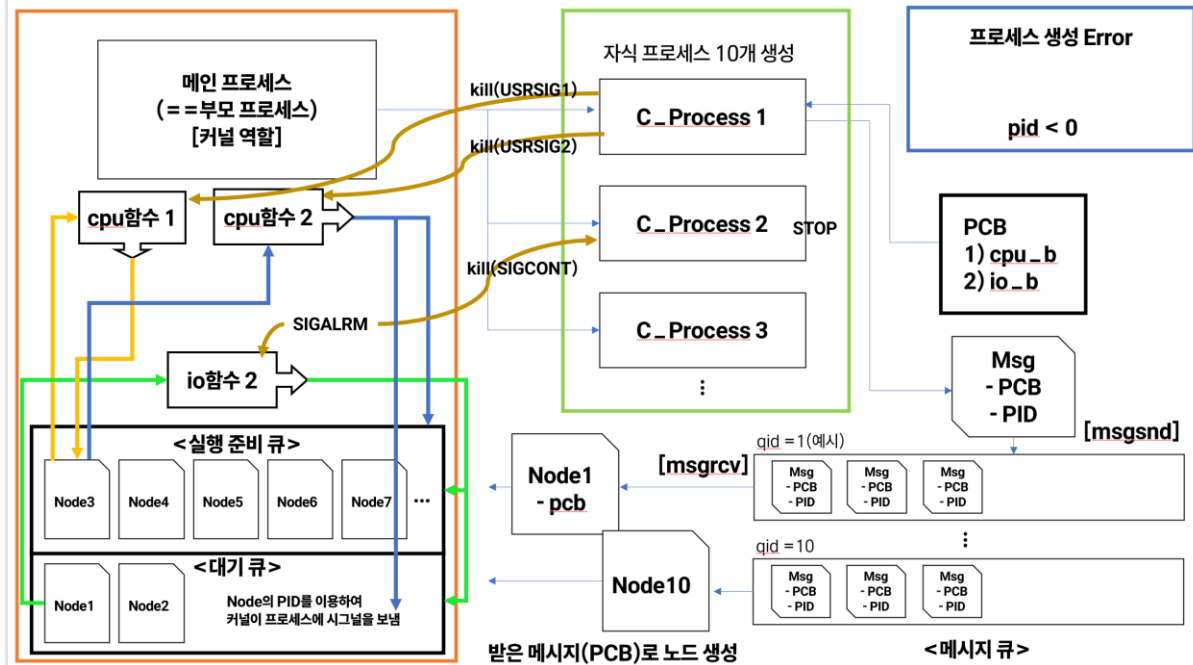
CPU 스케줄링

CPU 스케줄링은 운영 체제가 시스템의 CPU를 여러 프로세스 간에 효율적으로 할당하기 위해 사용하는 중요한 메커니즘이다. 다중 프로세스 시스템에서 CPU 스케줄링은 시스템 성능을 극대화하고 응답 시간, 처리율, 대기 시간을 최적화하는 데 필수적이다. 여러 프로세스 시스템에서 CPU 스케줄링이 필수적인 이유는 다음과 같다. 이전의 프로세스 상태에서 다뤘듯이 프로세스는 CPU작업과 IO작업을 하게 되는데 만일 CPU 작업이 끝났는데도 해당 CPU를 점유한 채로 IO작업을 하게 되면 비효율적이다. 따라서 만일 IO작업을 하게 될 시 다른 프로세스에 CPU 사용 권한을 넘겨 효율을 증가시켜야 한다. 주요 스케줄링 방식으로는 먼저 선점형과 비선점형으로 나눌 수 있다. 선점형이란, 어떤 프로세스가 CPU작업을 전부 끝내기 전에 다른 프로세스가 CPU 사용 권한을 얻어서 CPU작업을 할 수 있는 방식을 말한다. 반대로 비선점형이란, 어떤 프로세스가 자신의 CPU작업을 전부 끝내기 전까지 CPU사용을 독점하는 방식을 말한다.

	스케줄링 알고리즘	장점	단점
선점형	Round Robin	공정성 보장, 짧은 응답 시간	시간 할당량이 작으면 문맥 전환이 잦아져 오버헤드 증가
	SRTF	대기 시간 최소화	자주 선점이 발생해 문맥 전환 오버헤드 발생
비선점형	FCFS	간단하고 구현이 쉬움	대기 시간이 길어질 수 있음
	SJF	평균 대기 시간을 최소화	실행 시간을 미리 알기 어렵고, 긴 작업은 대기 시간이 길어질 수 있음

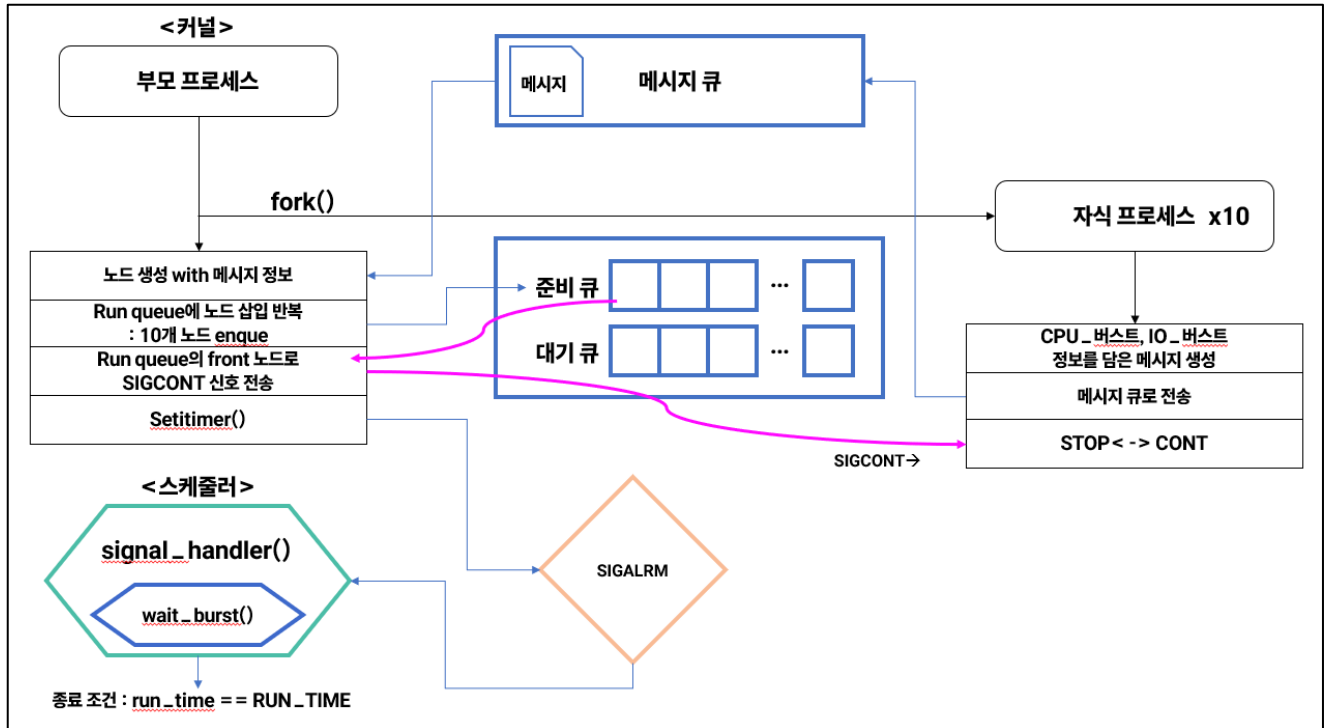
위의 표와 같이 각 스케줄링 방식들은 장점과 단점들이 존재한다. 따라서 운영체제에서 CPU 스케줄링 알고리즘을 선택할 때는 시스템의 성격과 사용자 요구 사항을 고려해야 한다. 예를 들어, 짧은 응답 시간이 중요한 인터랙티브 시스템에서는 Round Robin방식이 적합하고, 평균 대기 시간이 중요한 배치 시스템에서는 SJF 또는 SRTF가 적합할 수 있다. 또한 만일 프로세스의 우선순위가 있다면 우선순위 스케줄링 방식을 사용할 수도 있다.

스케줄링 프로그램 설계 및 구현 계획



[그림 11] 스케줄링 프로그램 설계 구상도 (1차)

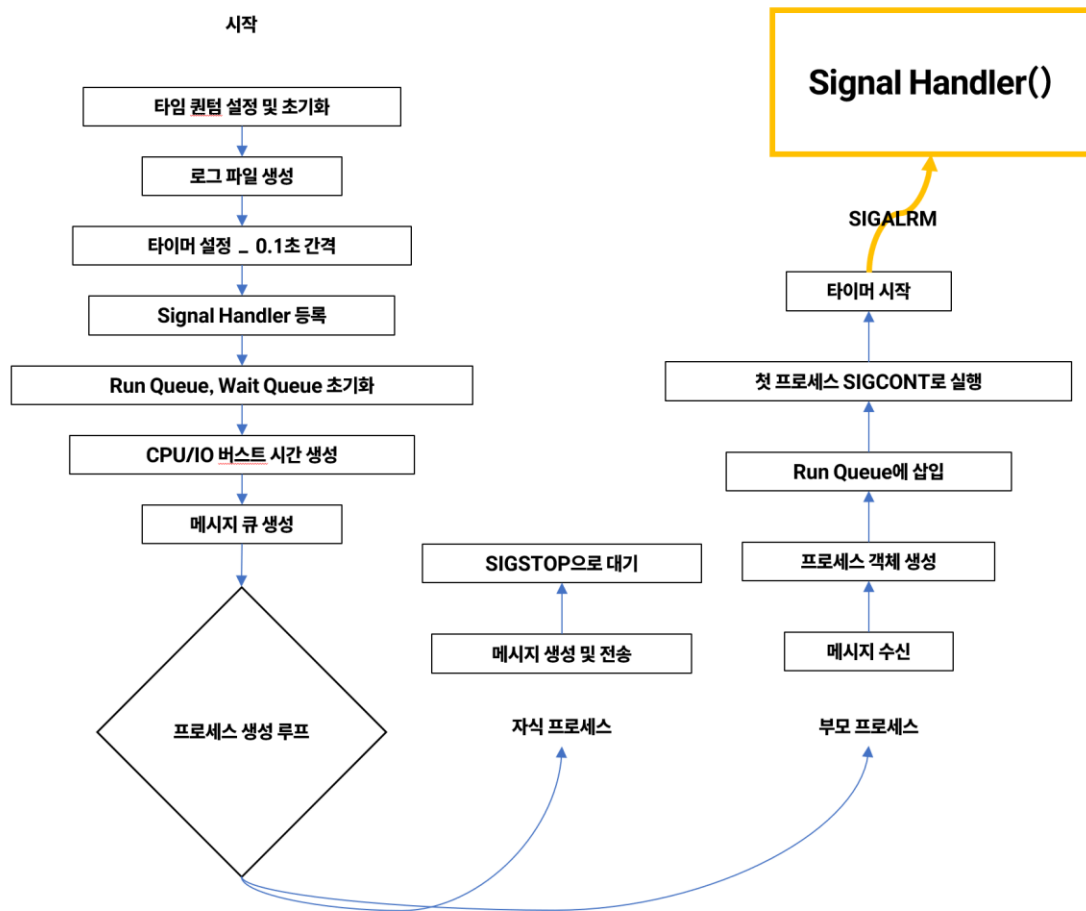
위의 [그림11]은 첫번째로 그려보았던 1차 스케줄링 프로그램 설계 구상도다. 특징으로는 각 프로세스마다 별개의 메시지 큐를 사용하는 것과 스케줄링 시스템이 3가지 함수로 구현된다는 것, 그리고 자식 프로세스와 부모 프로세스가 기본 적인 신호 외에 사용자가 정의한 신호인 SIGUSR1, SIGUSR2 신호 2가지를 추가로 사용한다는 점이 있다. 그러나 실제로 구현하기 위해 고민하던 중 좀 더 간단히 시뮬레이터를 구현할 수 있을 것 같아 조금 변경된 두번째 계획을 세우게 되었다. 변화된 두번째 구현 계획은 다음과 같다.



[그림 12] 스케줄링 프로그램 설계 구상도 (2차)

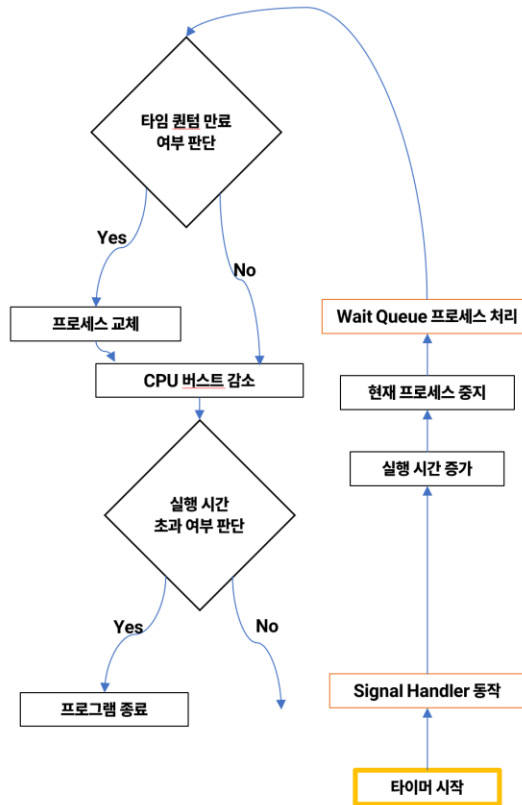
위의 [그림 12]를 통해 알 수 있듯이 2 차 설계 구상에서는 1 차와 달리 여러 자식 프로세스가 하나의 메시지 큐를 사용하고 있으며 사용자 정의 시그널은 사용하지 않는다. 또한 스케줄러는 기능별로 쪼개어 여러 함수가 담당하지 않게 하나의 덩어리 함수로 구현한다.

다음은 실제 코딩할 때 논리적으로 생각하여 작성할 수 있게끔 하기 위해 SW 글로벌공학 시간에 배운 내용을 적용하여 구상도를 기반으로 flow chart 를 그려 보았다. 첫 번째 자료의 마지막인 Signal Handler()는 두 번째 자료의 Signal Handler 동작으로 이어진다.

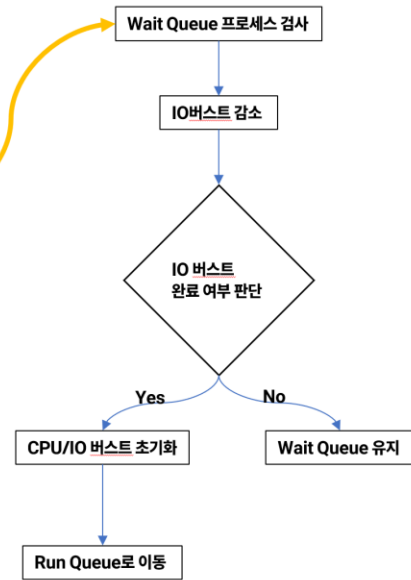


[그림 13-1] 2번째 구상도를 구현하기 위해 논리적인 흐름을 표현한 플로우 차트 (1)

타이머 핸들러 동작



Wait queue 처리



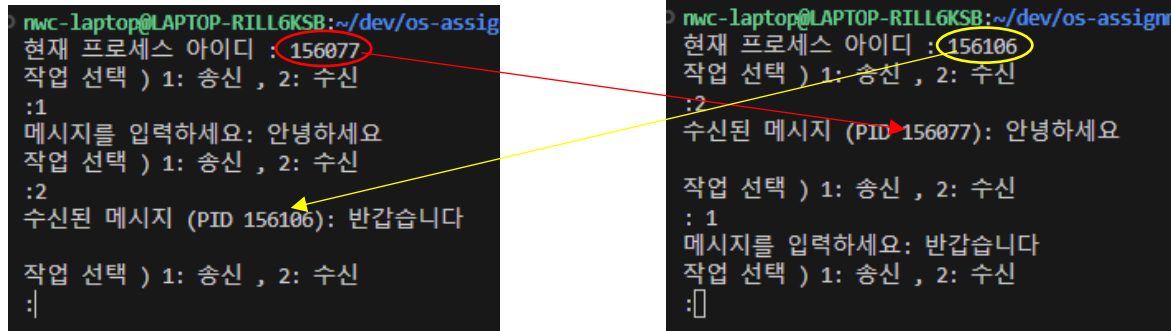
[그림 13-2] 2번째 구상도를 구현하기 위해 논리적인 흐름을 표현한 플로우 차트 (2)

이렇게 기본적인 설계와 계획을 마쳤으므로 이제 실제로 구현해보자.

스케줄링 프로그램 구현

필자의 경우 이번 과제를 진행하며 저번과 마찬가지로 처음 보는 라이브러리들을 사용하게 되어 본격적인 구현을 하기에 앞서 스케줄러 구현에 사용되는 기능들을 해당 기능들을 간단히 사용해 보며 중요 포인트들을 다루고 넘어가고자 한다.

1) IPC 통신 실습 _ 간단한 프로세스 간 채팅 프로그램을 만들어 실습해보았다. (chat.c)



```
nwc-laptop@LAPTOP-RILL6KSB:~/dev/os-assig
현재 프로세스 아이디 : 156077
작업 선택 ) 1: 송신 , 2: 수신
:1
메시지를 입력하세요: 안녕하세요
작업 선택 ) 1: 송신 , 2: 수신
:2
수신된 메시지 (PID 156106): 반갑습니다
작업 선택 ) 1: 송신 , 2: 수신
:|

nwc-laptop@LAPTOP-RILL6KSB:~/dev/os-assig
현재 프로세스 아이디 : 156106
작업 선택 ) 1: 송신 , 2: 수신
:2
수신된 메시지 (PID 156077): 안녕하세요
작업 선택 ) 1: 송신 , 2: 수신
: 1
메시지를 입력하세요: 반갑습니다
작업 선택 ) 1: 송신 , 2: 수신
:|
```

[그림 14] 동일한 프로그램을 동시에 실행 - 프로세스간 통신.

```
int main() {
    int select;
    int k = 1234;
    int qid = msgget(k, IPC_CREAT | 0666);
    pid_t pid = getpid();
```

[그림 15] 하나의 키를 사용하여 동일한 큐 아이디를 이용 _ chat.c

여기서는 위 [그림14],[그림15]에서 보여지듯 서로 다른 프로세스가 하나의 키를 사용하여 만들어진 큐 아이디의 메시지 큐를 이용하여 메시지를 주고 받는 모습이다. 이 경우 동일한 메시지 큐를 이용하기 때문에 프로세스 간 통신이 가능 한 것을 알 수 있다.

2) 시그널 실습 _ tick.c

```
void ticktock(int signo) {
    printf("tick-tock\n");
}

int main() {
    struct itimerval new_itimer;
    struct itimerval old_itimer;

    // 타이머의 반복 간격 설정 (1초)
    new_itimer.it_interval.tv_sec = 1;
    new_itimer.it_interval.tv_usec = 0;
    // 타이머의 초기 만료 시간 설정 (1초 뒤 시작)
    new_itimer.it_value.tv_sec = 1;
    new_itimer.it_value.tv_usec = 0;

    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = ticktock; // ticktock 함수를 시그널
    sigaction(SIGALRM, &sa, NULL);

    // 타이머 설정
    setitimer(ITIMER_REAL, &new_itimer, &old_itimer);

    // 무한 대기
    while (1);

    return 0;
}
```

[그림 16] sigaction 함수를 이용해 특정 신호 발생 시 임의의 함수를 호출

```
nwc-laptop@LAPTOP-RILL6KSB:~/dev/os-assignment/new/lee/lee-3/2024-os-proj1/my/laboratory$ gcc tick.c
nwc-laptop@LAPTOP-RILL6KSB:~/dev/os-assignment/new/lee/lee-3/2024-os-proj1/my/laboratory$ ./a.out
tick-tock
tick-tock
tick-tock
tick-tock
tick-tock
tick-tock
tick-tock
```

[그림 17] 1초 마다 ticktock 함수가 호출되어 출력

위의 그림에서 알 수 있듯 sigaction 함수를 이용하면 신호와 함수를 연결 지을 수 있다. 또한 kill 함수를 이용하여 프로세스를 특정하여 신호를 보낼 수도 있다. 스케줄러 구현에는 SIGALRM, SIGCONT, SIGSTOP가 사용될 예정이다.

3) 스케줄러 시뮬레이터 구현

```
else if (pid == 0) { // 자식 프로세스
    printf("[%d] %d번 자식 프로세스 생성 성공 || cpu: %d, io: %d\n",
           getpid(), i, cpu_burst[i], io_burst[i]);

    struct msgbuf snd_msg;
    snd_msg.mtype = i+1;
    snd_msg.idx = i;
    snd_msg.pid = getpid();
    snd_msg.cpu_burst = cpu_burst[i];
    snd_msg.io_burst = io_burst[i];

    if (msgsnd(msgq, &snd_msg, sizeof(snd_msg) - sizeof(long), 0) == -1) {
        perror("msgsnd failed");
        exit(1);
    }

    //자식 프로세스 : 부모 프로세스로부터 SIGCONT신호를 받아 재개
    while(1) {
        kill(getpid(), SIGSTOP);
        printf("**[%d] 자식 프로세스 실행중*\n", getpid());
    }
}
```

[그림 18] 자식 프로세스

```

else { // 부모 프로세스
    printf("[%d번] 자식 프로세스 생성 실행\n", pid);

    struct msgbuf rcv_msg;
    if (msgrcv(msgq, &rcv_msg, sizeof(rcv_msg) - sizeof(long), i+1, 0) == -1) {
        perror("msgrcv failed");
        exit(1);
    }
    printf("[ IPC ]   msg qid : %d, index : %d , cpu burst : %d , io burst : %d \n\n",
           msgq, rcv_msg.idx, rcv_msg.cpu_burst, rcv_msg.io_burst);

    //자식 프로세스로부터 받은 메시지를 이용하여 대응하는 노드 생성
    Process* new_proc = malloc(sizeof(Process));
    new_proc->pid = rcv_msg.pid;
    new_proc->idx = rcv_msg.idx;
    new_proc->io_burst = rcv_msg.io_burst;
    new_proc->remain_io_burst = rcv_msg.io_burst;
    new_proc->cpu_burst = rcv_msg.cpu_burst;
    new_proc->remain_cpu_burst = rcv_msg.cpu_burst;

    //새로 만든 노드 실행 큐에 삽입
    if (!isFull(&runq)) {
        enqueue(&runq, new_proc);
    } else {
        printf("Run queue is full. Cannot add new process.\n\n");
        free(new_proc);
    }
}

// 실행 큐의 front에 SIGCONT 신호 전송
if (!isEmpty(&runq)) {
    cur_proc = &runq.processes[runq.front];
    kill(cur_proc->pid, SIGCONT);
}
//---> 현재 cur_proc 프로세스는 pause해제 상태,
// 타이머 시작
setitimer(ITIMER_REAL, &new_itimer, &old_itimer);

while (1) {
    pause();
}

```

[그림 19] 부모 프로세스 주요 부분

```

//SIGALRM 신호마다 호출 : 기본적으로 cpu버스트를 관리하고, 추가적으로 waitq_burst()
//라운드 로빈 방식으로 설계한 스케줄러 함수
void signal_handler(int signo) {
    run_time++;
    count--;

    // 현재 실행 중인 프로세스가 있으면 중지
    if (cur_proc != NULL) {
        kill(cur_proc->pid, SIGSTOP);
    }

    print_qstate(run_time, &runq, &waitq);
    writeToFile(file, run_time, count, &runq, &waitq);
    printf("[time_quantum : %d] -----> count: %d\n", time_quantum, count);

    // waitq의 프로세스들 처리
    waitq_burst();

    // 타임 퀀텀 만료 또는 CPU 버스트 완료 체크
    if (count == 0 || (cur_proc && cur_proc->remain_cpu_burst == 0)) {
        if (!isEmpty(&runq)) {
            Process* proc = dequeue(&runq);

            if (proc->remain_cpu_burst > 0) {
                enqueue(&runq, proc);
            } else {
                enqueue(&waitq, proc);
            }
            count = time_quantum;
        }
    }

    // 현재 다루고 있는 프로세스의 cpu버스트 1 감소
    if (!isEmpty(&runq)) {
        cur_proc = &runq.processes[runq.front];
        cur_proc->remain_cpu_burst--;
        kill(cur_proc->pid, SIGCONT);
    }

    //실제 실행 시간이 목표 실행시간에 도달할 시 종료(6000초 == 10분, 프로세스당 평
    if (run_time == RUN_TIME) {

```

[그림 20] signal_handler()

```

//Wait 큐에 들어있는 모든 프로세스의 I/O 버스트 1 감소
void waitq_burst() {
    int initial_size = waitq.size;

    while (initial_size > 0) {
        Process* proc = dequeue(&waitq);
        initial_size--;

        proc->remain_io_burst--;

        if (proc->remain_io_burst == 0) { // 버스트 모두 소진 시 재할당(초기화)
            proc->remain_cpu_burst = proc->cpu_burst;
            proc->remain_io_burst = proc->io_burst;
            enqueue(&runq, proc);
        } else {
            enqueue(&waitq, proc);
        }
    }
}

```

[그림 21] waitq_burst

```

typedef struct msgbuf {
    long mtype;

    int pid;
    int idx;
    int io_burst;
    int cpu_burst;
} msgbuf;

typedef struct Process {
    pid_t pid;
    int idx;
    int cpu_burst;
    int io_burst;

    int remain_cpu_burst;
    int remain_io_burst;
} Process;

typedef struct Queue {
    Process processes[QUEUE_SIZE];
    int front;
    int rear;
    int size;
} Queue;

```

[그림 22] 구조체

[스케줄러 실행 결과]

```
nwc-laptop@LAPTOP-RILL6KSB:~/dev/os-assignment/new/lee/lee-3/os_hw/os_hw$ make run ARGS="30"
./sec_program 30
입력한 타임 쿼텀 값 : 30

[162049번] 자식 프로세스 생성 실행
[ IPC ]   msg qid : 229425, index : 0 , cpu burst : 36 , io burst : 191

[162049] 0번 자식 프로세스 생성 성공 || cpu: 33, io: 22
[162050] 1번 자식 프로세스 생성 성공 || cpu: 22, io: 33
[162050번] 자식 프로세스 생성 실행
[ IPC ]   msg qid : 229425, index : 1 , cpu burst : 22 , io burst : 33

[162051번] 자식 프로세스 생성 실행
[162051] 2번 자식 프로세스 생성 성공 || cpu: 23, io: 66
[ IPC ]   msg qid : 229425, index : 2 , cpu burst : 23 , io burst : 66

[162052번] 자식 프로세스 생성 실행
[162052] 3번 자식 프로세스 생성 성공 || cpu: 17, io: 108
[ IPC ]   msg qid : 229425, index : 3 , cpu burst : 17 , io burst : 108

[162053번] 자식 프로세스 생성 실행
[162053] 4번 자식 프로세스 생성 성공 || cpu: 8, io: 31
[ IPC ]   msg qid : 229425, index : 4 , cpu burst : 8 , io burst : 31

[162054번] 자식 프로세스 생성 실행
[162054] 5번 자식 프로세스 생성 성공 || cpu: 32, io: 195
[ IPC ]   msg qid : 229425, index : 5 , cpu burst : 32 , io burst : 195

[162055번] 자식 프로세스 생성 실행
[162055] 6번 자식 프로세스 생성 성공 || cpu: 7, io: 45
[ IPC ]   msg qid : 229425, index : 6 , cpu burst : 7 , io burst : 45

[162056번] 자식 프로세스 생성 실행
[162056] 7번 자식 프로세스 생성 성공 || cpu: 48, io: 54
[ IPC ]   msg qid : 229425, index : 7 , cpu burst : 48 , io burst : 54

[162057번] 자식 프로세스 생성 실행
[162057] 8번 자식 프로세스 생성 성공 || cpu: 22, io: 90
[ IPC ]   msg qid : 229425, index : 8 , cpu burst : 22 , io burst : 90

[162058번] 자식 프로세스 생성 실행
[162058] 9번 자식 프로세스 생성 성공 || cpu: 41, io: 114
[ IPC ]   msg qid : 229425, index : 9 , cpu burst : 41 , io burst : 114

-----
      모든 자식 프로세스 실행 준비 큐에 삽입 완료
```

[그림 23] 실행 결과

```
----- 스케줄링 시작 -----

[Information]
- 스케줄링 방식 : Round-Robin
- Total Run Time : 6000
- Time_quantum : 30

----- System Status at Time 1 -----

RUN QUEUE | P0[CPU:33] → P1[CPU:47] → P2[CPU:10] → P3[CPU:46] → P4[CPU:49] → P5[CPU:41] → P6[CPU:44] → P7[CPU:48] → P8[CPU: 2] → P9[CPU:35]
WAIT QUEUE | Empty

[time_quantum : 30] -----> count: 29
```

[그림 24] 스케줄링 설정 (cpu 버스트가 50 이하이므로 타임 쿼텀 또한 50보다 작아야 유의미)

```
----- System Status at Time 1144 -----

RUN QUEUE | P2[CPU:29] → P9[CPU:12] → P6[CPU:42] → P7[CPU:30]
WAIT QUEUE | P3[I/O:68] → P0[I/O: 7] → P1[I/O:12] → P8[I/O:121] → P4[I/O:22] → P5[I/O:49]

[time_quantum : 30] -----> count: 13
*[162185] 자식 프로세스 실행중*

----- System Status at Time 1145 -----

RUN QUEUE | P2[CPU:28] → P9[CPU:12] → P6[CPU:42] → P7[CPU:30]
WAIT QUEUE | P3[I/O:67] → P0[I/O: 6] → P1[I/O:11] → P8[I/O:120] → P4[I/O:21] → P5[I/O:48]

[time_quantum : 30] -----> count: 12
*[162185] 자식 프로세스 실행중*

----- System Status at Time 1146 -----

RUN QUEUE | P2[CPU:27] → P9[CPU:12] → P6[CPU:42] → P7[CPU:30]
WAIT QUEUE | P3[I/O:66] → P0[I/O: 5] → P1[I/O:10] → P8[I/O:119] → P4[I/O:20] → P5[I/O:47]

[time_quantum : 30] -----> count: 11
*[162185] 자식 프로세스 실행중*
```

```
----- System Status at Time 5999 -----

RUN QUEUE | P5[CPU:26] → P2[CPU:16] → P4[CPU: 5] → P3[CPU:12] → P1[CPU:21]
WAIT QUEUE | P7[I/O:15] → P9[I/O:82] → P6[I/O:123] → P8[I/O:117] → P0[I/O:82]

[time_quantum : 30] -----> count: 24
*[162188] 자식 프로세스 실행중*

----- System Status at Time 6000 -----

RUN QUEUE | P5[CPU:25] → P2[CPU:16] → P4[CPU: 5] → P3[CPU:12] → P1[CPU:21]
WAIT QUEUE | P7[I/O:14] → P9[I/O:81] → P6[I/O:122] → P8[I/O:116] → P0[I/O:81]

[time_quantum : 30] -----> count: 23

----- Program finished -----
```

[그림 25] 실행 과정 : 총 6000틱(600초) 동안 지속 → 프로세스 당 평균 1분 소요

```
===== Time: 94 (Quantum Count: 22) =====
```

```
Current Process: P4
```

```
|— CPU Burst Remaining: 38
```

```
|— I/O Burst Total: 25
```

```
Run Queue Status:
```

```
|— P4 (CPU: 38)
```

```
|— P5 (CPU: 23)
```

```
|— P6 (CPU: 44)
```

```
|— P7 (CPU: 4)
```

```
|— P8 (CPU: 8)
```

```
|— P9 (CPU: 4)
```

```
|— P2 (CPU: 6)
```

```
Wait Queue Status:
```

```
|— P0 (I/O: 19)
```

```
|— P1 (I/O: 24)
```

```
|— P3 (I/O: 174)
```

```
===== Time: 95 (Quantum Count: 21) =====
```

```
Current Process: P4
```

```
|— CPU Burst Remaining: 37
```

```
|— I/O Burst Total: 25
```

```
Run Queue Status:
```

```
|— P4 (CPU: 37)
```

```
|— P5 (CPU: 23)
```

```
|— P6 (CPU: 44)
```

```
|— P7 (CPU: 4)
```

```
|— P8 (CPU: 8)
```

```
|— P9 (CPU: 4)
```

```
|— P2 (CPU: 6)
```

```
Wait Queue Status:
```

```
|— P0 (I/O: 18)
```

[그림 26] 덤프 파일

사용 방법

구성

- Makefile
- msg.h
- queue.c
- sec.c

사용 방법

- make run ARGS="타임 쿼텀"
 - 예) make run ARGS = '30'
- make clean